TurtleBot3 Motion Server

Design, Setup, and Operation Guide

A browser-based teleoperation and camera-streaming system

`ttb-experiments`

August 6, 2026

Abstract

This document explains, from the ground up, a system that lets you **drive a TurtleBot3 robot from a web browser** while watching its camera in real time. It assumes *no prior knowledge of ROS* (the Robot Operating System) or robotics. It covers the underlying concepts, the software that was written, the network and configuration issues that were solved along the way, and complete step-by-step instructions to start, use, and stop the system.

Contents

1 Introduction

Imagine a small wheeled robot with a camera on top. This project lets you open a web page on your laptop, see through the robot's camera, and press buttons (or use the keyboard) to drive it around — forward, backward, and turning — all wirelessly. We call this **teleoperation**, or "teleop" for short: operating a machine from a distance.

The whole thing is one Python program, `motion_server.py`, that runs on a laptop. It does three jobs at once:

1. Receives the live camera images from the robot and shows them in the browser.
2. Sends your button presses to the robot as movement commands.
3. Serves a web page (the graphical user interface, or **GUI**) that ties it all together.

The equipment. There are two computers involved:

- The **TurtleBot3** (model "Burger") — a small robot whose brain is a Raspberry Pi. It has two motors, a spinning distance sensor (a lidar), and a USB camera.
- Your **laptop** — where the web server runs and where you open the browser.

They talk to each other over Wi-Fi.

2 A Crash Course for Total Beginners

This section explains just enough background to understand everything else. If you already know ROS, skip to Section 3.

2.1 What is ROS 2?

ROS 2 (Robot Operating System 2) is not really an operating system — it is a *toolkit* for building robot software. Its most important idea is that a robot's software is split into many small programs called **nodes**, and these nodes talk to each other by passing **messages**.

2.2 Topics: the robot's "radio channels"

Nodes communicate over named channels called **topics**. Think of a topic as a radio frequency with a name, like `/cmd_vel` or `/image`.

- A node that **publishes** to a topic is a broadcaster.
- A node that **subscribes** to a topic is a listener.

Any number of listeners can tune in. In our system:

- The robot's camera node **publishes** pictures on the topic `/image/compressed`. Our server **subscribes** to it.
- Our server **publishes** movement commands on the topic `/cmd_vel`. The robot's motor node **subscribes** to it.

2.3 Messages: Twist and images

Every message on a topic has a fixed shape (a "type"):

- A movement command is a **Twist** message. It carries a *linear* speed (how fast to go forward/backward, in metres per second) and an *angular* speed (how fast to spin, in radians per second). Sending `linear=0.1, angular=0` means "drive forward gently." Sending all zeros means "stop."

- A camera picture is an **Image** (or **CompressedImage**, which is the same picture squeezed smaller — like a JPEG — so it travels faster over Wi-Fi). Our robot sends the compressed kind.

2.4 How two computers find each other: the Domain ID

ROS 2 nodes discover each other automatically over the network using a system called **DDS**. To keep separate robot projects from interfering, every node belongs to a **ROS_DOMAIN_ID** — a number from 0 to 232. *Two computers can only see each other's topics if they share the same ROS_DOMAIN_ID.* In this project both the robot and the laptop use **domain 203**. (A mismatch here was one of the first bugs we hit — see Section 8.)

3 System Architecture

Figure 1 shows how data flows. The camera images travel *from* the robot *to* the browser; the drive commands travel the other way.

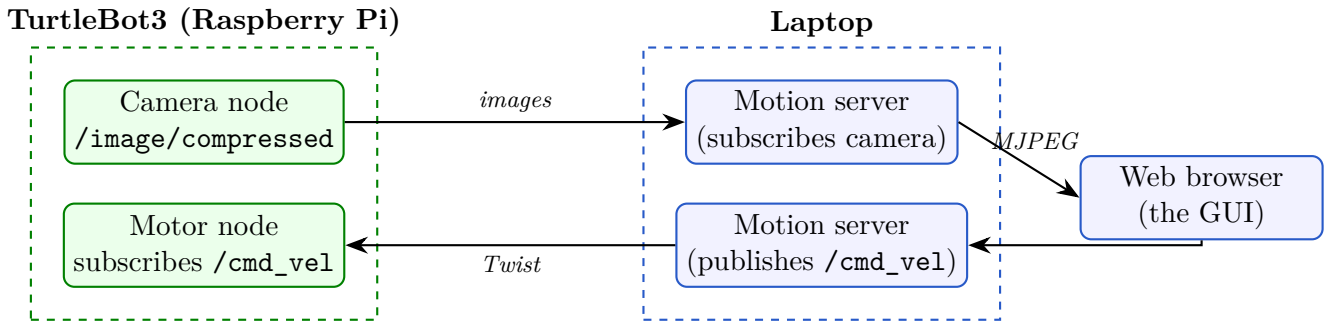

Figure 1: Data flow. Left: the robot. Right: the laptop. Camera images flow right; drive commands flow left. Wi-Fi carries the ROS 2 traffic between them.

The two dashed boxes are the two computers. Inside the laptop, the single `motion_server.py` program plays several roles: it subscribes to the camera, re-serves those frames to the browser as a video stream, and turns your clicks into `/cmd_vel` messages.

4 Hardware and Network Configuration

Table 1 lists every setting the system depends on. All of these live in a single `.env` file so they are easy to change (see Appendix A).

5 ROS 2 Topics and Interfaces Reference

Section 2.4 introduced topics as "radio channels." This section is the complete reference for the channels this robot actually broadcasts.

A point that confuses newcomers: *topics are not defined in configuration files.* There is no file you can open that lists them. A topic springs into existence when a running node asks for it in code — for example, the camera node creates its channel with the single line

```
self.publisher_ = self.create_publisher(CompressedImage, '/image/compressed', 1)
```

(see `robot/.../image_publisher.py`, line 38). So the authoritative list of topics comes from asking the *live* robot, not from reading a file. Table 2 is exactly that: the output of `ros2 topic`

Setting	Value	Meaning
Robot IP	`192.168.68.100`	Address of the robot on the Wi-Fi
Laptop IP	`192.168.68.113`	Address of the laptop
Robot login	`ubuntu`	SSH username on the robot
ROS_DOMAIN_ID	`203`	Shared "channel group" (Sec. 2.4)
Robot model	`burger`	The TurtleBot3 variant
Lidar model	`LDS-01`	The distance sensor type
Camera topic	`/image/compressed`	Where images are published
Command topic	`/cmd_vel`	Where drive commands go
Max linear speed	`0.22` m/s	Burger's top forward speed
Max angular speed	`2.84` rad/s	Burger's top turning speed
Server port	`8000`	Web address is `localhost:8000`

Table 1: System configuration.

`list` on a running system, with each type resolved.

Topic	Message type	Published by
`/cmd_vel`	`geometry_msgs/msg/Twist`	**this app** (laptop → robot)
`/image/compressed`	`sensor_msgs/msg/CompressedImage`	`image_publisher` (custom)
`/odom`	`nav_msgs/msg/Odometry`	`diff_drive_controller` (read by **this app**)
`/scan`	`sensor_msgs/msg/LaserScan`	`hlds_laser_publisher`
`/imu`	`sensor_msgs/msg/Imu`	`turtlebot3_node`
`/magnetic_field`	`sensor_msgs/msg/MagneticField`	`turtlebot3_node`
`/battery_state`	`sensor_msgs/msg/BatteryState`	`turtlebot3_node`
`/joint_states`	`sensor_msgs/msg/JointState`	`turtlebot3_node`
`/sensor_state`	`turtlebot3_msgs/msg/SensorState`	`turtlebot3_node`
`/tf`, `/tf_static`	`tf2_msgs/msg/TFMessage`	`robot_state_publisher`
`/robot_description`	`std_msgs/msg/String`	`robot_state_publisher`

Table 2: Every topic on the running system (`ROS_DOMAIN_ID=203`).

Of these eleven, the motion server touches exactly **three**: it subscribes to `/image/compressed` for the video feed and to `/odom` to measure distance and angle goals (Section 7.3), and publishes to `/cmd_vel`. Everything else comes free with the standard bringup and is there if you want to extend the project — `/scan` for obstacle avoidance, `/battery_state` for a charge warning.

5.1 Topics versus services

A **topic** is one-way broadcast: the sender publishes and never learns who listened. A **service** is a two-way request/response, like a phone call — you ask a question and wait for an answer.

Services *are* defined in files, which is why the distinction matters here. The robot package contains one, `srv/Motion.srv`, which declares a request of two floats and a response of a success flag plus a message. It belongs to the upstream Jetson AI stack and **is not used by this app** — the web GUI drives the robot purely by publishing `Twist` messages on `/cmd_vel`.

5.2 Inspecting topics yourself

```
export ROS_DOMAIN_ID=203
ros2 topic list # every topic currently on the network
ros2 topic type /cmd_vel # what message type it carries
ros2 topic echo /odom # print messages as they arrive
```

```
ros2 topic hz /scan # measure how fast they arrive
ros2 node list # which programs are running
```

If `ros2 topic list` shows only `/parameter_events` and `/rosout`, it is almost always a **stale ROS daemon**, not a network fault — and it will hide topics even on the robot itself. Fix it with `ros2 daemon stop`, or add `--no-daemon` to the command. Note that `ros2 topic hz` does *not* accept `--no-daemon` on Humble: it exits with a usage error that looks deceptively like "no data." Discovery also needs 20–30 seconds to fully settle after launch, so an incomplete list immediately after starting the robot is normal.

6 What Was Built: The Motion Server

The entire application is the file `motion_server.py`. Here is what each part does, in plain language.

6.1 Camera streaming

The server subscribes to the robot's camera topic. Each incoming compressed image is decoded into a picture. A special web address, `/video`, streams these pictures to the browser one after another (a technique called **MJPEG**), which the browser shows as a live video. The program can handle both "raw" and "compressed" image types and detects which one automatically.

6.2 Driving the robot

When you press a direction, the browser calls the address `/cmd` with a requested linear and angular speed. The server clamps those numbers to the robot's safe maximums and remembers them as the "target velocity."

6.3 Safety: the dead-man's switch

A robot that keeps moving after you lose connection is dangerous. To prevent this, the server re-sends the target velocity 20 times per second, but if no fresh command has arrived in the last **0.4 seconds** it automatically sends *stop*. In practice this means: the robot moves only while you are actively holding a button or key; the instant you release — or your browser or Wi-Fi drops — it halts. This is called a **dead-man's switch**.

6.4 The web GUI

Figure 2 shows the interface. On the left is the live camera. On the right is the control panel:

- A 3×3 grid of labelled buttons: *Forward, Backward, Turn Left, Turn Right*, the four diagonals, and a large red **STOP** in the centre.
- A live velocity read-out (v and ω).
- Sliders to set how fast forward driving and turning are.
- A latency read-out (explained in Section 6.5).

You can also drive with the keyboard: **W/A/S/D** or the arrow keys, and **Space** or **X** to stop. Press-and-hold to move; release to stop.

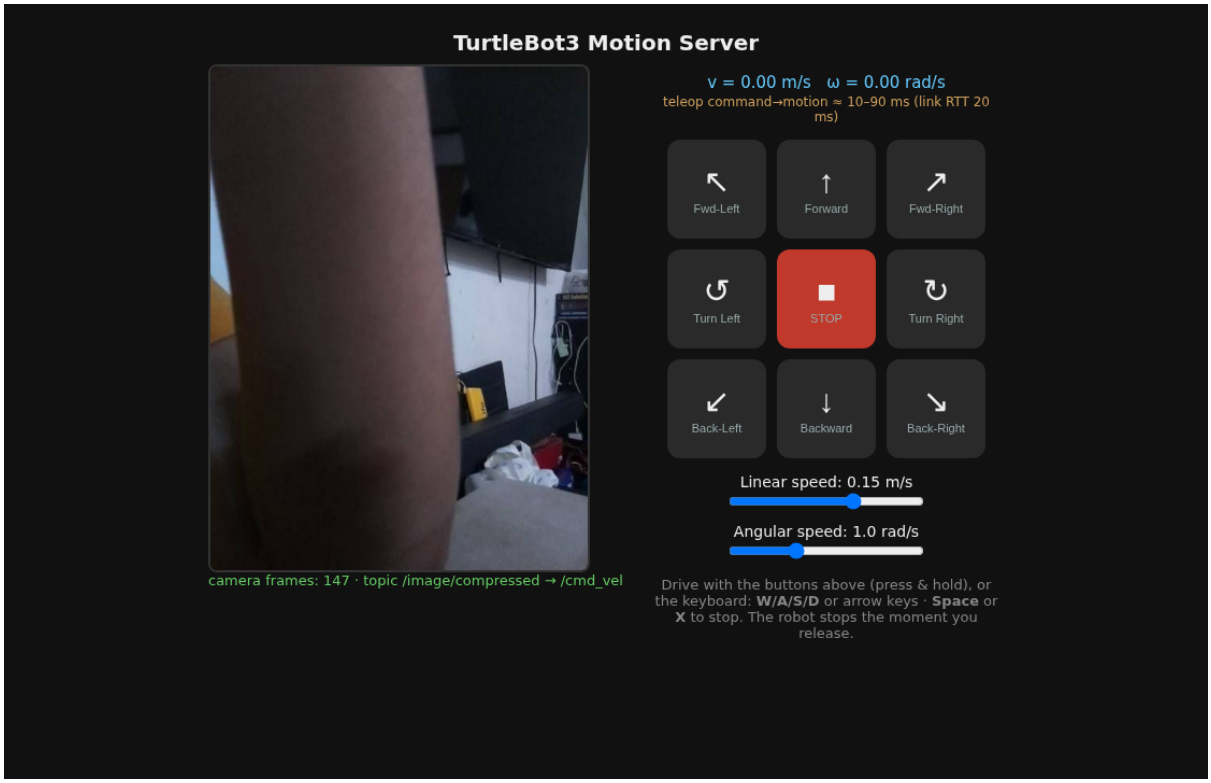

Figure 2: The web GUI. Camera feed on the left; full teleop control panel on the right with directional buttons, STOP, speed sliders, and the latency read-out.

6.5 Teleop latency read-out

The amber line under the velocity read-out estimates how long it takes for a button press to reach the robot as motion. It is built from a live measurement of the network **round-trip time** plus fixed allowances for internal processing — see Section 10 for the full explanation.

7 Natural-Language Control with a Local LLM

Holding a key is precise but wordless. This section adds a second way to drive: typing an instruction in plain English — *"move forward for 3 seconds"*, *"move forward 1 meter"*, *"rotate right 30 degrees"* — and having the robot carry it out. The language model runs **on the laptop**, so no text ever leaves the machine.

An instruction can specify how *long* to move, how *far* to move, or how far to *turn*. The first is open-loop timing; the other two are closed-loop and measured against the robot's odometry, which is the subject of Section 7.3. Several instructions can also be chained with commas and run one after another — Section 7.4.

7.1 Why a typed command needs a timer

This is the crux of the design, and it follows directly from the dead-man's switch in Section 6.3. Holding a key works because the browser sends a fresh command on every press; the robot keeps moving only while those commands keep arriving, and stops 0.4 s after they cease.

A typed instruction is a *single event*. Send it once and the robot lurches for 0.4 s and dies. So a natural-language command is executed by a small background worker that re-asserts the requested velocity every 0.1 s for the requested duration, then stops. It feeds the dead-man's

switch deliberately, for a bounded window.

Crucially, the dead-man's switch is left *completely untouched* underneath. If that worker thread crashes, or the whole server is killed mid-motion, no more commands arrive and the robot halts within 0.4 s anyway. The new feature sits on top of the existing safety net rather than replacing it.

7.2 The model only interprets; it never drives

The model's sole job is to turn a sentence into a small, fixed record:

```
{"action": "forward", "mode": "distance", "value": 1, "speed": "normal"}
```

`action` is restricted to six values (`forward`, `backward`, `rotate_left`, `rotate_right`, `stop`, `unknown`) by a JSON schema enforced during decoding, so the model physically cannot emit anything else. The result is then re-validated in Python and every number is clamped before it can reach a motor.

`mode` tags what unit `value` carries: `duration` means seconds, `distance` means metres, `angle` means degrees. One tagged number is used rather than three separate optional fields, for a practical reason — under schema-constrained decoding an "optional" field is a coin flip, so all three would end up mandatory, and a 3-billion-parameter model handed three mutually exclusive numbers will dutifully fill in all of them, inventing a duration for *"move forward 1 meter"* purely because the slot exists.

The order of the fields in the schema is also deliberate. Decoders emit properties in the order the schema lists them, so `mode` is placed *before* `value`: the model commits to the unit before it writes the number. Reversed, it picks a number blind and then rationalises a unit for it.

That last step is not a formality. During development both candidate models were asked to *"drive forward for 3 hours"*: one returned a duration of 10 800 seconds, the other 1 800 seconds, and one of them classified it as a perfectly ordinary drive command. Table 3 lists the guards that turn that into a harmless 10-second nudge.

Guard	**Effect**
Duration cap	Clamped to `NL_MAX_DURATION` (default 10 s)
Distance cap	Clamped to `NL_MAX_DISTANCE` (default 2 m)
Angle cap	Clamped to `NL_MAX_ANGLE` (default 360°)
Chain budget	Sequence refused over 5 steps / 3 m / 720° / 120 s
Chain aborts	One failed step abandons every step after it
Mid-chain stop	Refused; a `stop` must stand on its own
Speed clamp	Velocities clamped to `MAX_LIN` / `MAX_ANG`
Fixed action list	Anything outside the six actions is refused
Auto-stop	Every motion ends by itself; no "drive forever"
Goal timeout	Closed-loop goals also carry a wall-clock deadline
Progress watchdog	<2 mm or <1° for 1.5 s aborts the motion
Odometry watchdog	`/odom` silent for 0.5 s ⇒ stop
No odometry	Distance/angle refused outright, never estimated
STOP wins	STOP, **Space** or **X** aborts a typed motion
Manual override	**W/A/S/D** takes control from a running command
Dead-man backstop	Server dies mid-motion ⇒ robot stops in 0.4 s
Model unreachable	Falls back to a refusal, never to motion

Table 3: Safety guards on natural-language commands. All are enforced in code, not by asking the model nicely.

7.3 Distance and angle: measuring instead of guessing

The speed sliders and a duration together imply a distance — speed multiplied by time. It is tempting to stop there: to turn *"move forward 1 meter"* into $1 \div 0.154 = 6.5$ seconds of driving and call it done. That is *open-loop* control, and it is wrong in a way that cannot be tuned out. The robot accelerates rather than jumping to speed, the motors have a deadband, the wheels slip, and a tired battery turns slower than a fresh one. Errors of 10–20 % are normal, and nothing in the loop ever notices.

Instead the server subscribes to `/odom`, the odometry the robot already publishes 20 times a second, and *measures*. For a distance goal it records the starting position and watches the straight-line displacement grow; for an angle goal it watches the heading. When the remaining error falls inside a tolerance band, it stops.

7.3.1 Easing into the goal

A naïve version — drive at full speed until the goal is reached, then stop — overshoots badly, and the reason is worth spelling out. Between the instant the robot is actually at the goal and the instant its wheels stop, the following must happen: the next odometry sample has to arrive (up to 50 ms), cross the Wi-Fi (5–20 ms), be noticed by the control loop (up to 50 ms), be published on the next command tick (up to 50 ms), cross back, and finally be acted on by the motors (30–80 ms). That is roughly **150 ms** of unavoidable lag. At normal speed the robot travels 2 cm — or turns **17°** — in that time.

The fix is to slow down before arriving. The commanded speed is scaled down proportionally over the last 10 cm (or 25°), so by the time the stop decision is made the robot is already crawling, and 150 ms of lag costs millimetres instead of centimetres.

There is a floor under that ramp, and it matters: a TurtleBot3 below roughly 0.01 m/s simply buzzes without moving. The commanded speed is never allowed below about twice that. This produces a genuine tension — a higher floor overshoots, a lower one stalls short of the goal and burns the timeout — and it is the floor, not the tolerance, that ultimately bounds the error.

7.3.2 Turning past half a circle

Odometry reports heading as an angle between $-\pi$ and $+\pi$, which wraps. Subtracting a start heading from a current one naïvely means a 270° turn appears to fold back on itself and the goal is never reached — the robot would spin until the timeout. So each incoming sample's *change* in heading is unwrapped and accumulated into a running total that is free to pass π. At 20 Hz the largest genuine change between samples is about 0.14 rad, far from the π boundary, so the correction is never ambiguous. A jump larger than 2 rad is not a rotation at all but an odometry reset, and is flagged as such rather than silently corrupting the total.

7.3.3 Why this cannot mean "drive forever"

Section 6.3 promised that every motion ends by itself. A goal that stops when the robot *arrives* threatens exactly that promise, because a robot that cannot arrive would never stop. Three independent endings are therefore in place: reaching the goal; a progress watchdog that aborts after 1.5 s of no measurable movement; and a wall-clock timeout of roughly three times the ideal duration plus a margin. The 0.4 s dead-man's switch sits underneath all three.

None of these watchdogs catches a robot that has been *picked up*. TurtleBot3 odometry comes from wheel encoders, so wheels spinning in the air report flawless progress and a lifted robot will cheerfully "complete" a 1-metre move. The watchdogs catch *blocked* and *odometry-dead*, not *airborne*. This is a real limitation — and, conveniently, what makes testing on blocks so effective.

7.3.4 How accurate is it?

Expect about **±2 cm and ±5°** on a hard floor; carpet is worse. Note carefully what that is measured against: the robot's own wheel odometry, which itself drifts 1–3 % through slip. It is *not* a claim about absolute heading. Over a full 360° spin, odometry may report exactly 360.0 while the true heading is several degrees off. Tuning the tolerances to compensate for slip would be a mistake — that is a calibration problem, not a control problem.

7.4 Chaining several instructions

Commas (or *then*, or semicolons) join instructions into a sequence, run strictly in order, each beginning only once the previous one has finished:

```
move forward 0.5 m, turn right 90 degrees, move forward 0.3 m
```

7.4.1 Splitting happens before the model, not inside it

The obvious approach — ask the model for a *list* of steps — is the wrong one here. The text is split into clauses in Python and each clause is sent to the model on its own, for three reasons.

The decisive one is that failures stay **attributable**. Holding the fragment lets the server say *step 2 of 3 ("go to the kitchen"): not a driving command I understand.* Had the model chosen the decomposition, there would be no span of the user's own words to quote back at them.

Second, the prompt and schema from the previous section are left untouched, so the single-instruction behaviour they were tested against is unchanged by construction rather than by hope. Third, under schema-constrained decoding the *length* of an array is not reliably constrained, so a step limit would become a check applied after the fact to however many steps the model felt like emitting, instead of a property of the user's own punctuation.

The splitter has one subtlety worth stating: a fragment containing no movement word is treated as a *qualifier* and glued back onto its neighbour. This is what keeps *"move forward 1 m, slowly"* a single instruction. Note also that a bare *and* is deliberately not a separator — *"go forward a meter and a half"* must survive intact.

7.4.2 One thread, one cancel

The whole sequence runs on a single worker thread sharing a single cancellation flag. This is a safety property, not a tidiness one.

The tempting alternative — let each step, as it finishes, launch the next — races the STOP path. Stopping works by raising the cancel flag and then waiting briefly for the worker to exit. If a dying worker spawns its successor, that wait can return with a *brand new* motion already running, and the robot carries on. The STOP button would appear to do nothing. Keeping the whole sequence inside one cancellable worker means the existing STOP button, the keyboard, and a typed *halt* all abort the entire sequence without any of them needing to know that sequences exist.

7.4.3 A failed step abandons the rest

If any step ends for any reason other than reaching its goal — timeout, blocked wheels, odometry loss, the STOP button — the remaining steps are discarded rather than attempted.

This is the whole reason the feature needs care. Suppose *"forward 0.5 m, turn right 90°, forward 0.3 m"* and the turn stops at 41°. Carrying on would drive the final leg 49° away from where it was meant to go — confidently, and in a direction nobody asked for. A sequence is only meaningful while every step before it did what it said.

7.4.4 The pause between steps

Steps are separated by a deliberate 0.35 s pause with zero velocity commanded explicitly. It is not there for braking — the ramp of Section 7.3 hands off at the speed floor, so the robot is physically stopped in well under 50 ms.

It is there because the *measurement* lags the robot. Odometry is the last sample *received*, up to a 20 Hz period plus wireless transport old. A step that snapshots its starting pose while the robot is still moving measures its goal from a position the robot has already left. The pause lets several fresh samples arrive after the robot is genuinely still. It is also kept below the 0.4 s dead-man's timeout, so a healthy sequence never relies on that backstop.

7.4.5 Budgets for the whole sequence

Every cap in Table 3 was per *motion*. Three steps of two metres each would clear all of them individually and drive six metres. A sequence therefore carries its own budget — 5 steps, 3 m of path, 720°, 120 s worst case — and is **refused** rather than trimmed when it exceeds one.

Refusing breaks symmetry with the per-step caps on purpose. A capped step has a repair the user can see in their own sentence ("capped from 5 m"). Trimming a sequence to fit has no such reading: which step should lose out? The result would be a robot ending up somewhere the user could not have predicted from what they typed.

Note the distance budget bounds *path length*, not displacement: *"forward 2 m, backward 2 m"* spends 4 m of it and finishes where it started. For a room-sized safety argument, path length is the right thing to bound.

Chaining is sequential convenience, not a path-accurate trajectory. Each step measures from where the robot *believes* it is when that step starts, so errors compound: a 5° heading error left over from one turn rotates the whole of the next leg's displacement, putting a 0.3 m leg about 2.6 cm off to the side, and it grows with the length of the sequence. Four 90° turns will not reliably close a square. A path the robot genuinely follows needs a map and a localiser; this is dead reckoning with good manners.

7.5 What it costs you

Parsing takes roughly **2–4 seconds** on an RTX 3060 with `qwen2.5:3b` — the delay between pressing Enter and the robot moving. The first command after an idle period can take around 16 seconds while the model is loaded into memory; the server passes `keep_alive` to ollama to avoid paying that repeatedly. For reflex-speed driving, keep using the keyboard; this is a different, more conversational way to work.

7.6 Trying it without moving the robot

The `/nl` endpoint accepts `?dry_run=1`, which parses and clamps the instruction and reports what *would* happen, without sending any motion command. This is the safe way to check the model's understanding:

```
curl -s -X POST 'http://localhost:8000/nl?dry_run=1' \
     -H 'Content-Type: application/json' \
     -d '{"text":"move forward 5 meters"}'
# -> {"action":"forward","mode":"distance","value":2.0,"capped":true, ...}
```

Dry runs work with the robot switched off entirely, so phrasings can be developed without it. A second server started on a command topic the robot does not subscribe to is inert in a different and useful way:

```
python3 motion_server.py --cmd-vel-topic /cmd_vel_test --port 8001 \
       --image-topic /nonexistent --image-type raw --enable-nl
```

Nothing moves, but `/odom` is real — so every closed-loop goal ends in the progress watchdog. That is not a limitation of the test, it *is* the test: it proves the backstops fire, and it times them.

For chained commands this instance is more useful still. An all-duration sequence is open-loop, so it *runs to completion* with the robot standing still — a genuine end-to-end test of the sequencing, in which the total elapsed time also confirms that the pause between steps really happened. Mixing in a single distance or angle step then produces a real mid-sequence failure, which is how you check that the steps after it are abandoned rather than attempted.

Natural language is a convenience layer, not an autonomy layer. The robot has no idea what is in front of it — there is no obstacle avoidance on this path. It will happily drive into a wall if you ask it to. Keep watching the robot, keep a hand near STOP, and test with the wheels off the ground first.

8 Problems Encountered and How They Were Solved

Getting the system working surfaced several real-world issues. They are documented here because they are common and instructive.

Nothing was visible at first. The robot was powered on but its ROS software had not been started. Powering on a robot is not the same as launching its programs — the motor and camera nodes must be started explicitly.

Domain ID mismatch. The robot's start-up file (`.bashrc`) set the ROS_DOMAIN_ID *five* times with conflicting values; the last one won, leaving the robot on domain 0 while the laptop was on 203. Because of the rule in Section 2.4, they could not see each other. The file was cleaned to set a single, consistent value (203).

Missing lidar model. Launching the robot failed with `KeyError: 'LDS_MODEL'` because the lidar type was not set in the environment. Setting `LDS_MODEL=LDS-01` fixed it.

SSH dropping on launch. Every time the robot software started, the remote terminal connection reset (a Wi-Fi hiccup under the sudden burst of network traffic). The fix was to launch the robot programs *detached*, so they keep running even if the terminal disconnects.

Multicast worry. The home Wi-Fi (a `192.168.68.x` network, typical of Google/Nest routers) was suspected of blocking the "multicast" traffic ROS uses to discover nodes. A unicast

fall-back configuration (`fastdds_unicast.xml`) was prepared, but testing showed multicast actually worked, so the default is used.

9 How to Run Everything

This is the complete operating procedure.

9.1 One-time setup

```
# Copy the configuration template and fill in your values
cp .env.example .env
nano .env # set ROBOT_IP, ROBOT_PASSWORD, ROS_DOMAIN_ID, etc.
```

9.2 Step 1 — Start the robot's software

This starts the motor driver and the camera on the robot, over SSH:

```
./scripts/robot_start.sh
```

It launches everything on the correct domain and waits for it to come up. You can confirm the robot's topics are visible from the laptop with:

```
export ROS_DOMAIN_ID=203
ros2 topic list # you should see /cmd_vel, /image/compressed, /scan, ...
```

9.3 Step 2 — Start the web server

```
./run_server.sh
```

9.4 Step 3 — Open the GUI and drive

Open `http://localhost:8000` in a browser (or `http://192.168.68.113:8000` from your phone on the same Wi-Fi). Drive with the buttons or keyboard. *Watch the robot while you drive.*

9.5 Step 4 — Shut down

```
# Stop the web server: press Ctrl-C in its terminal, or:
pkill -f motion_server.py

# Stop the robot's ROS nodes (robot stays powered on):
./scripts/robot_stop.sh
```

9.6 Step 5 — Power off the robot (optional)

Always shut the Raspberry Pi down cleanly before cutting power, to avoid corrupting its memory card:

```
ssh ubuntu@192.168.68.100 'sudo poweroff'
# wait ~20 s for the green light to stop blinking, then flip the power switch
```

10 Understanding Teleop Latency

Latency is the delay between doing something and seeing the effect. For teleop, it is the time from pressing a button to the wheels actually turning.

The most important and variable part is the **Round-Trip Time (RTT)**: how long a small network packet takes to travel from the laptop to the robot *and back*. The server measures this live (it appears as "link RTT" in the GUI). A typical value on this setup is about 20 ms; if the Wi-Fi degrades, this number climbs and driving feels sluggish.

A command only needs to travel *one way* (laptop → robot), which is about half the RTT. The GUI's estimate therefore adds up:

$$\text{latency} \approx \underbrace{\tfrac{1}{2}\,\text{RTT}}_{\text{network, one way}} + \underbrace{50\,\text{ms}}_{\text{command cycle (20 Hz)}} + \underbrace{\sim 30\,\text{ms}}_{\text{motor response}} .$$

For a 20 ms RTT this gives roughly 10–90 ms. Note this is a well-grounded *estimate*: only the RTT is measured directly; the other terms are known constants of the system.

11 Troubleshooting

Symptom	Likely cause and fix
`ros2 topic list` shows nothing from the robot	Robot software not started (run `robot_start.sh`), or domain mismatch (both must be `ROS_DOMAIN_ID=203`).
GUI says "waiting for camera" but the robot drives	Wrong camera topic; check `ros2 topic list` and `IMAGE_TOPIC` in `.env`.
Camera shows but robot will not move	Wrong command topic, or the robot's motor node is not running.
Robot launch fails: `KeyError: 'LDS_MODEL'`	Set `LDS_MODEL` (e.g. `LDS-01`) before launching.
SSH disconnects when starting the robot	Expected Wi-Fi hiccup; the scripts launch detached so the nodes survive. Just reconnect and check.
Driving feels laggy	Check the "link RTT" in the GUI; a high value means poor Wi-Fi.

Table 4: Common problems.

12 Repository Structure and Security

```
ttb-experiments/
  motion_server.py # the whole application (node + server + GUI)
  run_server.sh # start the server on the laptop (reads .env)
  scripts/
    robot_start.sh # start the robot's ROS nodes over SSH
    robot_stop.sh # stop the robot's ROS nodes
  fastdds_unicast.xml # optional fall-back for multicast-blocking networks
  .env.example # configuration template (safe to share)
  .env # YOUR real config incl. password (git-ignored!)
  robot/ # reference snapshot of the ROBOT-side files
    start_all.sh # launches bringup + camera on the robot
    turtlebot3_image_motion/
      turtlebot3_image_motion/image_publisher.py # the camera node
```

```
    srv/Motion.srv # a service definition (unused by this app)
    package.xml / CMakeLists.txt
docs/
  turtlebot3_motion_server.tex / .pdf # this document
  images/gui.png
```

About `robot/`. Those files run on the robot, not the laptop, and are copied here only so this repository documents the whole system. They are a snapshot of commit `54fb8a2` of a separate project, `https://github.com/SuperMadee/AI361-MEX3`, which remains the source of truth — the robot builds from its own clone in `~/ros2_ws`, so editing the copies here changes nothing on the robot. Only the parts this project depends on were copied; the upstream package's Jetson-side AI stack (roughly 15,000 lines) is deliberately not duplicated.

Security note. The robot's SSH password and other real values live only in `.env`, which is listed in `.gitignore` and is therefore *never* committed or pushed to GitHub. Only `.env.example`, containing placeholders, is shared. Never put real passwords in a public repository — automated bots scan GitHub for leaked credentials within minutes.

A Environment Variable Reference

Variable	Description
`ROBOT_IP`	Robot's IP address on the Wi-Fi.
`LAPTOP_IP`	Laptop's IP address.
`ROBOT_USER`	SSH username on the robot.
`ROBOT_PASSWORD`	SSH password (secret; kept only in `.env`).
`ROS_DOMAIN_ID`	Shared discovery domain; must match on both machines.
`TURTLEBOT3_MODEL`	Robot variant (`burger`).
`LDS_MODEL`	Lidar model (`LDS-01`).
`IMAGE_TOPIC`	Topic the camera publishes on.
`IMAGE_TYPE`	`compressed`, `raw`, or `auto`.
`CMD_VEL_TOPIC`	Topic drive commands are published on.
`ODOM_TOPIC`	Odometry topic read to measure distance/angle goals.
`MAX_LIN`	Max linear speed (m/s).
`MAX_ANG`	Max angular speed (rad/s).
`SERVER_PORT`	Web server port.
`ENABLE_NL`	`1` to enable natural-language driving (Sec. 7).
`LLM_URL`	ollama base URL (default `http://localhost:11434`).
`LLM_MODEL`	Model used to parse commands (`qwen2.5:3b`).
`NL_MAX_DURATION`	Hard cap in seconds on any single typed motion.
`NL_MAX_DISTANCE`	Hard cap in metres on any single typed distance goal.
`NL_MAX_ANGLE`	Hard cap in degrees on any single typed angle goal.
`GOAL_TIMEOUT_MAX`	Ceiling in seconds on a closed-loop goal's timeout.
`NL_MAX_STEPS`	Most steps allowed in one chained sequence.
`NL_MAX_CHAIN_DISTANCE`	Cap in metres on total path length per sequence.
`NL_MAX_CHAIN_ANGLE`	Cap in degrees on total rotation per sequence.
`NL_MAX_CHAIN_SECONDS`	Cap on a sequence's worst-case running time.

B Command Cheat-Sheet

```
# --- laptop ---
./run_server.sh # start the web GUI server
pkill -f motion_server.py # stop it
export ROS_DOMAIN_ID=203; ros2 topic list # what the robot is publishing

# --- robot (over SSH) ---
./scripts/robot_start.sh # start motors + camera
./scripts/robot_stop.sh # stop them
ssh ubuntu@192.168.68.100 'sudo poweroff' # clean power-off
```